Zerve

MarshMcLennan

Presents IIT Bombay's

TECHFEST

22nd -24th December 2025

Asia's Largest Science & Technology Festival

A SIMULATED PARADIGM

Designing Risk Intelligence, Not Just Models

Team ID: Zerv-250475

Team Name: Zebra

Team members: Om Chauhan, Lakshit Vedant

Zerve

Problem Reality Check

Understanding the True Nature of the Problem and the Practical Constraints That Shape the Solution.

Dataset: **Health Insurance Claim**

Why this was NOT a standard binary classification problem

Key observations

1. Extreme class imbalance
2. Evaluation based on Normalized Gini, not accuracy / AUC
3. Predictions used for ranking decisions, not labels
4. Output probabilities must be monotonic & calibrated

Hidden Challenge

Good ranking ≠ good probability estimates

I optimized ranking stability + probability trustworthiness.

Goal

Build a ML Model that estimates customer-level insurance ***claim risk*** from feature data, enabling customers to be ***ordered*** by risk and supporting pricing differentiation and risk management decisions, with claim outcomes represented as ***0/1 targets***.

Zerve

Baseline & What Failed

Three critical stages that transform ideas into better results

Plain Boosting

Improved local metrics, but quickly saturated — unable to capture minority-class ranking depth under extreme imbalance.

Aggressive SMOTE

Artificial balance amplified noise, distorted class geometry, consistently hurt ranking stability on validation data.

Single Model

LightGBM and XGBoost peaked well individually, but exhibited unstable rankings under small distribution changes.

Heavy Features

Feature count increased, but signal density dropped — leading to overfitting without meaningful Gini gains.

Advanced Stacking

Added 5 layer stack with AdaBoost, Gradient Boost, but it lowered the performace by 13%.

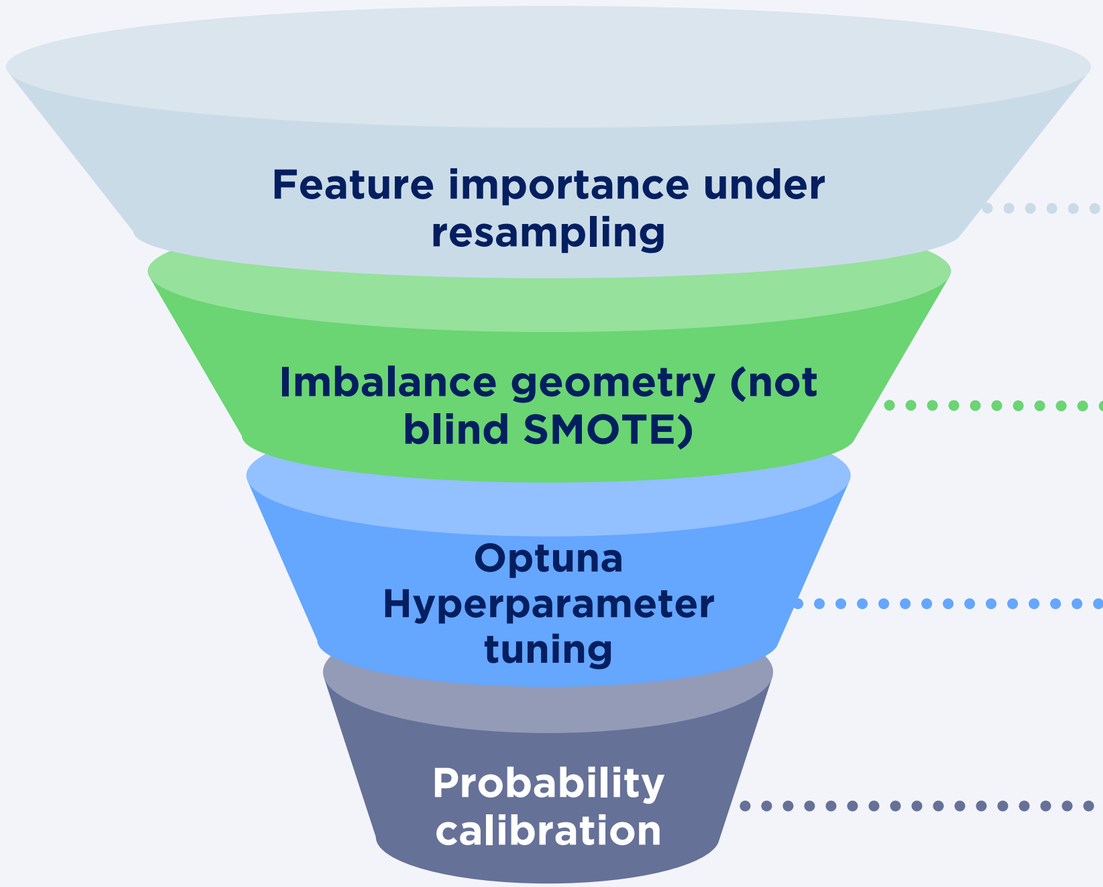
Raw Probabilities

Produced overconfident risk scores rankings looked strong, but probability magnitudes were unreliable

Zerve

Pipeline Philosophy

Designing for Signal, Stability, and Scale.



Features selected after resampling to avoid imbalance-biased importance.

Class ratios tuned empirically to preserve ranking gradients.

Bayesian optimization directly maximizing normalized Gini.

Monotonic score transformation for reliable risk magnitudes.

Zerve

Feature Engineering 101

Engineering Features That Respect the Data.

Missingness Modeling (8 features)

What exactly I added

- 4 explicit missing flags (feature_8,38,39,45)
- 2 row-level missing statistics (missing_count, missing_ratio)

Why this matters

- Missingness patterns were non-random
- Let model learn absence → risk correlation

These features could be

- (employer info, lifestyle flags)
- (medical tests only done for high-risk profiles)

Feature Interactions (24 features)

In insurance-like data, interactions often map to:

- Exposure × frequency
- Usage × duration
- Health metric × age proxy

Example:

- feature_24 * feature_29. → exposure × utilization

Why this helped ranking

- Claim risk is not additive
- These interactions let shallow trees capture second-order risk
- Improved separation among mid-risk customers (where Gini is most sensitive)

Zerve

Feature Engineering 101

Engineering Features That Respect the Data.

Ratio-Based Normalization (4 features)

What exactly I added

- 4 ratios to normalize magnitude bias
- Used numerical stability constant (1e-5)

Why this matters

- Raw magnitudes dominate splits in tree models
- Ratios enforce scale invariance
- Stabilized ranking across customers with very different absolute values
- Reduces variance-induced misranking caused by scale imbalance.

Polynomial Risk Shaping (12 features)

What exactly I added

- 4 core numeric features
- For each → squared, cubed, sqrt

Why this matters

- Allowed smooth non-linearity
- Avoided deep trees / overfitting
- Squared/cubed terms capture risk acceleration
- Sqrt dampens extreme values

Zerve

Feature Engineering 101

Engineering Features That Respect the Data.

Binary Density Aggregation (3 features)

What exactly I added

- Across 17 binary indicators
- Computed: binary_sum ,binary_mean, binary_std

Why this matters

- Single flag ≠ high risk
- Concentration of flags signals systemic risk
- Standard deviation captures inconsistency in behavior

Smoothed Target Encoding (14 features)

What exactly I added

- 14 categorical features
- Smoothed target mean encoding
- Smoothing factor = 10

Why this matters

- Prevented artificial spikes
- Improved monotonicity of predicted risk

Stage	Feature Count
Raw dataset	50
After engineering	**109**
After selection	**(-26.6%) 80**

Our Method

1. Applied optimized SMOTE + under-sampling strategy
2. Trained LightGBM on the resampled dataset
3. Recomputed feature importances under corrected class geometry
4. Selected Top-80 features that retained predictive power post-resampling

Zerve

Imbalance Is Geometry, Not Ratio

Class Imbalance Shapes the Learning Space.

Why Resampling Was Critical

1. Severe class imbalance distorted ranking gradients
2. Blind balancing degraded Gini despite improving class recall
3. Goal was ranking quality, not class parity

Original data: 96.36% no claim, 3.64% claim (1:26 ratio)

Ratio	Gini
1:1	0.2687
1:2.5	0.2728
1:5.0	0.2761
1:7.0	0.2759

Option 1: Only SMOTE

Introduced excessive synthetic points
Amplified noise in minority regions
Consistently reduced validation Gini

Option 2: Only Under-sampling

Discarded informative majority samples
Collapsed decision boundaries
Lost ranking resolution

Option 3: SMOTE + Under-sampling

Preserved real majority structure
Added controlled minority signal
Allowed precise control over class geometry

Zerve

Validation Strategy

"Ensuring Fair and Representative Model Evaluation.

Original Dataset

Total samples: 476,169 Class 0: 458,814 (96.36%) Class 1 : 17,355 (3.64%)

Why Hold-Out Validation?

- Training performance alone is misleading
- Models may memorize training patterns (overfitting)
- Validation data simulates unseen future customers

Random Split Risk

Training data may have 3.2% claims

Validation data may have 4.5% claims → not representative

Stratified Split Benefit

Preserves 3.64% claim ratio in both training and validation sets

Ensures fair, stable, and comparable evaluation

Split	Training Size	Validation Size
90 / 10	428,552	47,617
80 / 20	**380,935**	**95,234**
70 / 30	333,318	142,851

Zerve

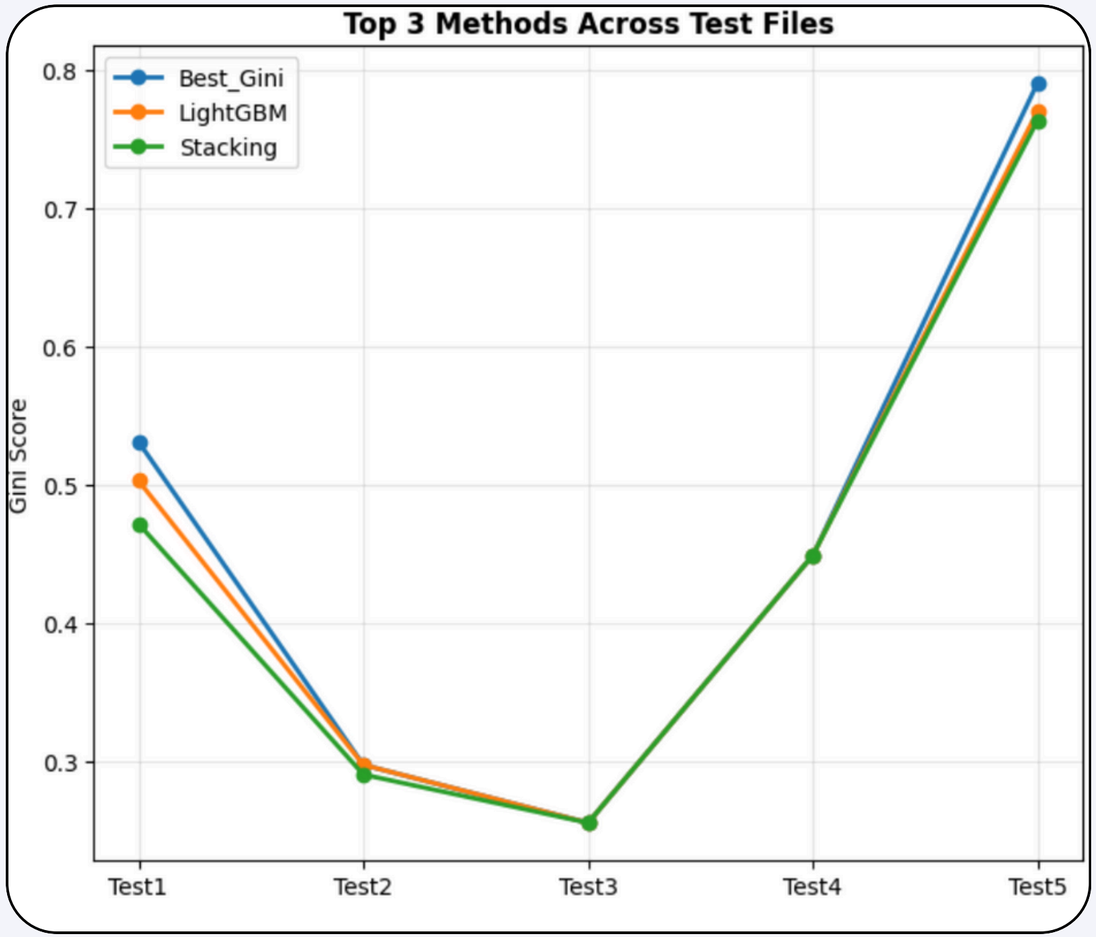
Testing Strategy

Testing That Reflects Real-World Performance.

Controlled Test-Set Construction for Realistic Evaluation

- Minority class oversampled to 20% using SMOTE to create a stable positive pool
- Multiple test sets sampled at 1.5%–5.5% prevalence from this pool
- Feature distributions preserved; only class ratios varied
- Enabled evaluation under realistic prevalence shift, not a single static split

Imbalance Percentage	Normalized Gini
1.50%	0.471503
2.50%	0.290285
3.50%	0.255385
4.40%	0.448773
5.50%	0.763113



Zerve

Models Stacking Ensemble

An ensemble of heterogeneous tree models was used to stabilize rankings and improve generalization

LightGBM

01

Leaf-wise gradient boosting decision trees

Captures sharp local patterns and feature interactions

Fast learner, sensitive to engineered features

XGBoost

02

Level-wise boosted trees with strong regularization

Produces stable and robust predictions

Controls overfitting under imbalance

CatBoost

03

Ordered boosting with built-in leakage prevention

Handles mixed feature types effectively

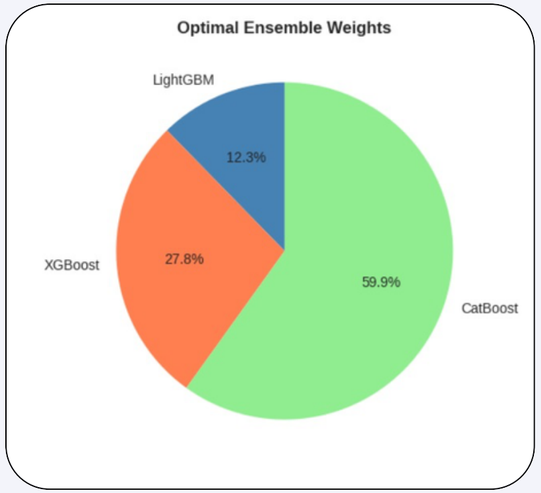
Produces smooth, well-ordered risk estimates

Zerve

Optimized Weighted Ensemble

A Learnable Combination of Complementary Models.

Combined predictions from LightGBM, XGBoost, and CatBoost treated ensemble as a linear meta-model over model probabilities

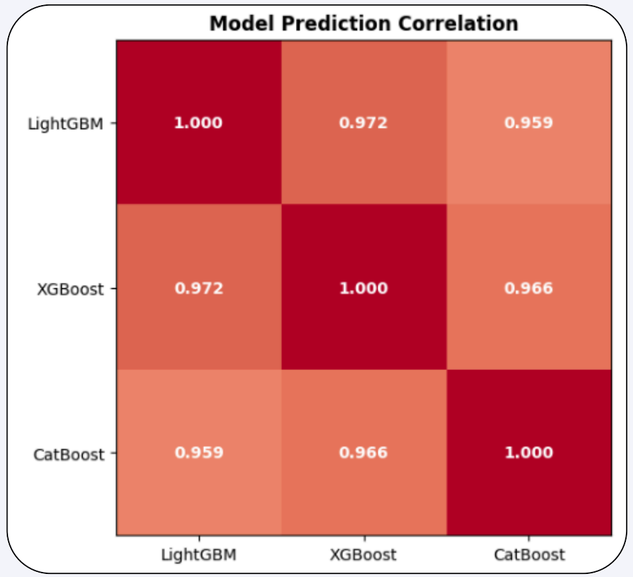


How it was optimized

- Ran ~50 trials with different weight combinations
- Evaluated each combination using Normalized Gini on validation data
- Selected weights that maximized ranking performance, not accuracy

Why this works

- Different models capture complementary risk patterns
- Weight optimization emphasizes reliable models and down-weights noisy ones
- Improves ranking stability while preserving diversity



Key Observations

- LightGBM: 0.1231
- XGBoost: 0.2778
- CatBoost: 0.5991

Zerve

Optuna Hyperparameter Tuning

Searching the Space Where Performance Actually Lives

Optimization Strategy

- 50 Optuna trials per model
- Tuned LightGBM, XGBoost, CatBoost
- Objective evaluated on validation Gini (OOF-style holdout)

Hyperparameter focus

- `Model capacity: num_leaves, max_depth, depth`
- `Learning dynamics: learning_rate, n_estimators`
- `Regularization: reg_alpha, reg_lambda, min_child_weight`
- `Sampling: feature_fraction, bagging_fraction, subsample`

Model	Before Tuning Gini	After Optuna Gini
LightGBM	0.2536	**0.277**
XGBoost	0.2433	**0.2817**
CatBoost	0.2627	**0.2834**
Ensemble	0.2606	**0.2872**

Zerve

Optuna Hyperparameter Tuning

Step by step methodology for building comprehensive and actionable project roadmaps

LightGBM Hyperparameters:

- objective: 'binary'
- metric: 'auc'
- boosting_type: 'gbdt'
- num_leaves: 27
- learning_rate: 0.030296008261125624
- feature_fraction: 0.5290361557648319
- bagging_fraction: 0.7079963887287573
- bagging_freq: 6
- max_depth: 6
- min_child_samples: 70
- reg_alpha: 1.967797180857751
- reg_lambda: 0.3766766852178698
- min_gain_to_split: 0.26652288380010525
- n_estimators: 727
- random_state: 42

Performance: OF Gini = 0.659

XGBoost Hyperparameters:

- objective: 'binary:logistic'
- eval_metric: 'auc'
- max_depth: 6
- learning_rate: 0.01283280231643027
- subsample: 0.5400197610813512
- colsample_bytree: 0.7010031274205561
- min_child_weight: 10
- reg_alpha: 0.5802044702069478
- reg_lambda: 2.605779041098784
- gamma: 0.24778006032550942
- n_estimators: 834
- random_state: 42
- tree_method: 'hist'

Performance: OOF Gini = 0.661

CatBoost Hyperparameters:

- iterations: 1339
- learning_rate: 0.010340023492710213
- depth: 8
- l2_leaf_reg: 6.042890350565001
- border_count: 71
- random_seed: 42
- eval_metric: 'AUC'
- early_stopping_rounds: 50

Performance: OOF Gini = 0.660

Zerve

Isometric Calibration

Probability Calibration with Isotonic Regression.

The Problem

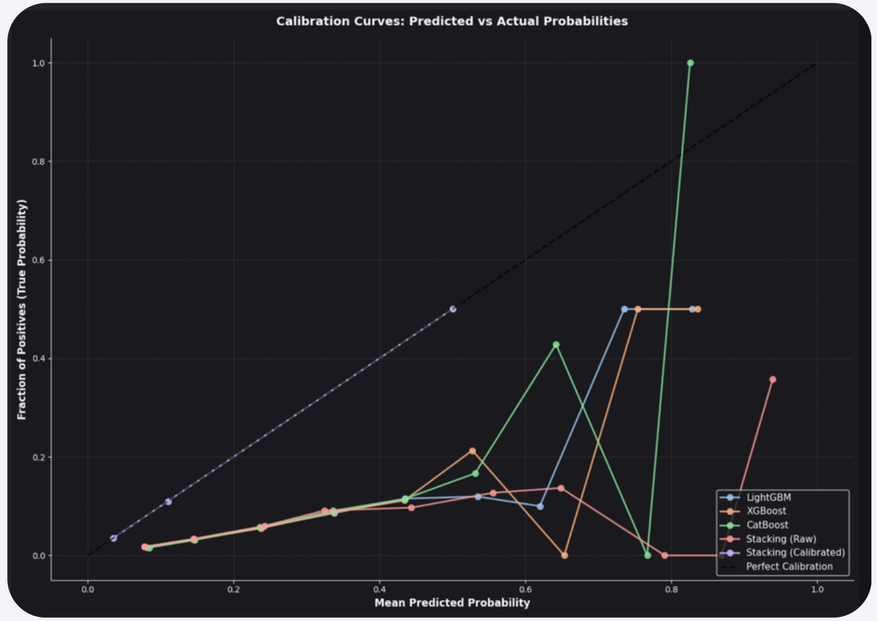
- Gradient boosting models optimize loss, not probability
- Outputs are often overconfident (e.g., 0.8 ≠ 80% real risk)
- Ranking may look good, but risk magnitude is misleading

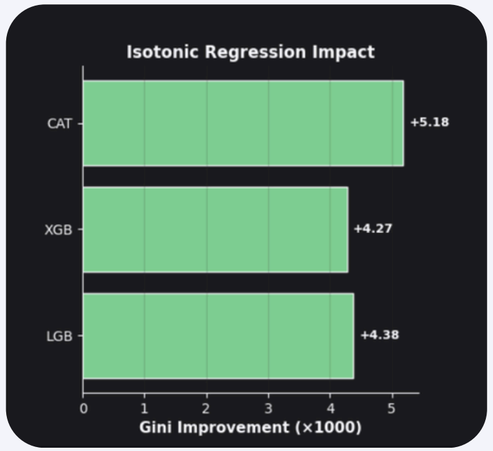
Why Isotonic Regression? (Technical Reason)

- Does not assume sigmoid shape (unlike Platt scaling)
- Ideal when base model already ranks well (Gini-focused task)
- Adjusts probability scale, not decision boundaries

Why Calibration Helped Gini

- Although calibration is monotonic:
- It stretches mid-risk regions
- Reduces score ties
- Improves cumulative gain curve shape





Zerve

Final Results

Optimized for stability, not just score.

Performance Impact

- Baseline Gini: 0.2773. (basic boosting, no calibration, no geometry control)
- Final Calibrated Ensemble: 0.285+. (+0.0078 absolute Gini gain)

What Actually Made the Difference

✓ Feature Selection after Imbalance

✓ Imbalance Geometry Testing (not blind 1:1)

✓ Calibration-Aware Modeling

✓ Diverse Models, Weighted Intelligently

✓ Ranking-First Design

Model	Validation Gini	Performance Summary
LightGBM	0.277097	Strong baseline
XGBoost	0.281795	Competitive
CatBoost	0.283442	Strongest single model
Ensemble	**0.287223**	**Best Score**